

U.T.2 Administración remota.

Sumario

1 Administración remota.....	2
2 Instalación.....	3
3 Configuración.....	3
4 Clientes SSH.....	5
4.1 OpenSSH.....	5
4.1.1 Iniciar sesión en un servidor con un puerto diferente.....	5
4.1.2 Ejecución de un solo comando en un servidor remoto.....	6
4.2 Putty.....	7
5 Autenticación de clave pública.....	7
6 Autenticación de en ssh mediante clave pública exclusivamente.....	7
6.1 Identificación del servidor.....	7
6.2 Claves del servidor.....	8
6.3 Claves en el cliente.....	10
6.4 Identificación del cliente.....	12
6.4.1 Cliente openssh.....	12
6.4.1.1 Eliminar o cambiar la frase de contraseña en una clave privada.....	12
6.4.2 Copia de su clave SSH pública a un servidor con SSH-Copy-ID.....	13
6.4.3 Copia de su clave SSH pública a un servidor sin SSH-Copy-ID.....	13
7 Utilidades adicionales.....	13
7.1 Transferencia de ficheros.....	13
8 Autenficación de dos factores en SSH.....	14
8.1 Instalación y configuración de Google Authenticator.....	15
8.2 Configurar SSH.....	15
8.3 Configurar la autenticación.....	16
8.4 Instalación de la aplicación Google Authenticator en el móvil.....	17
8.5 Pruebe la autenticación de dos factores.....	18

1 Administración remota

Es común que el administrador del servidor no se siente físicamente junto a él para configurarlo, sino que sus tareas de mantenimiento las realice a distancia sentado sobre un ordenador distinto. Ello hace necesario disponer de una terminal remota que habilite una sesión interactiva y que ofrezca al administrador las mismas posibilidades que la terminal física.

Tradicionalmente, este servicio lo vino desempeñando telnet, hasta que la inseguridad de la red lo volvió poco recomendable y fue sustituido por SSH, que cifra la comunicación y sortea sus problemas de seguridad, además de tener algunas otras funcionalidades muy útiles e interesantes.

Aunque el servicio sigue estando disponible (el paquete telnetd) no es en absoluto recomendable instalarlo, dado que adolece de dos problemas fundamentales de seguridad, comunes a todas las comunicaciones no cifradas:

- **Intercepción de la comunicación**

Al establecerse una comunicación no cifrada entre dos extremos, un tercero malicioso puede interceptar los paquetes y leerlos, extrayendo la información contenida en ellos. Esta información puede, por supuesto, incluir usuario y contraseña, que como el resto no van cifrados.

- **Suplantación de identidad**

También existe la posibilidad de suplantar la identidad del servidor, de modo que el cliente crea que se está conectado al servidor real, cuando en realidad se conecta a uno falso. Esto puede permitir al suplantador obtener también las claves de acceso al pedírselas al cliente.

Ambos problemas pueden evitarse mediante técnicas de cifrado híbrido, por lo que en 1995, un finlandés llamado Tatu Ylönen desarrolló la primera versión del protocolo SSH1, en principio, software libre. Apenas dos años después de su creación, en 1997, el protocolo fue propuesto como estándar de la IETF. La licencia, no obstante, cambió, por lo que en 1999 el equipo de OpenBSD comenzó la implementación más habitual del servidor, OpenSSH, que es la que traen las distribuciones de linux en su sistema de paquetería.

El protocolo SSH ataja los dos problemas de seguridad gracias a su cifrado híbrido que supone que:

1. Tras el establecimiento de la comunicación, el servidor envía al cliente su clave pública para que este pueda cotejarla y comprobar que no existe suplantación de identidad. Por supuesto, cotejarla implica conocerla de antemano y esto sólo es posible si el cliente se ha conectado anteriormente alguna vez. Si es la primera vez (y no media el crédito de ninguna entidad certificadora), el cliente tiene que hacer

el acto de fe de creer que el servidor es quien dice ser, razón por la cual todos los clientes advierten del hecho y obligan al usuario a aceptar implícitamente la clave.

2. Con la clave pública del servidor el cliente cifra una clave simétrica de sesión, que envía al servidor. Este al recibirla la descifra y, una vez la clave de sesión se halla en ambos extremos, puede comenzarse una comunicación absolutamente cifrada que impide sacar provecho de la intercepción de la comunicación.
3. Asegurado el cifrado, el cliente se identifica en el servidor, bien a través de usuario y contraseña, bien a través del uso de certificado.
- 4.

2 Instalación

Linux descontados adicionales, dispone de tres paquetes relacionados con openssh:

1. openssh-client, que instala el cliente y que viene de serie incluso en la instalación más básica.
2. openssh-server, que instala el servidor en sí.
3. El metapquete ssh, que instala los dos anteriores.

Así pues, basta con hacer:

```
# apt-get install ssh
```

para proceder a la instalación del servidor, la cual, además, dejará el servidor en funcionamiento. Como en el caso de cualquier otro servicio, pueden usarse tanto invoke-rc.d como systemctl para manipularlo.

3 Configuración

Todo la configuración del servidor se encuentra dentro del directorio /etc/ssh:

Las claves pública y privada del servidor con nombre ssh_host_*. Hay varias parejas porque se generan con varios algoritmos de cifrado¹.

El fichero ssh_config, que es, en realidad, el fichero general de configuración del cliente.

El fichero sshd_config, que será el que tengamos que editar si queremos alterar la configuración.

El fichero está profusamente comentado y, además, tiene página de manual propia por lo que aquí sólo se comentarán algunas directivas que se consideran de interés. En este fichero las directivas tienen su valor predeterminado, por lo que comentar o dejar descomentada una línea, tiene el mismo efecto, si no se modifica además el valor:

- **Port**

Port 22

Es el puerto de escucha. Lo habitual es que el servicio SSH escuche en el puerto 22 y que este valor no se cambie. Es posible, sin embargo, que este puerto sea inaccesible desde redes remotas fiscalizadas por un cortafuegos

- **PermitEmptyPasswords**

PermitEmptyPasswords no

Por razones de seguridad lo habitual es impedir que usuarios con contraseña vacía puedan acceder al servidor.

- **PermitRootLogin**

PermitRootLogin prohibit-password

Define si el administrador puede directamente ingresar en el servidor. Lo aconsejable es impedirlo, porque en cuanto un servidor SSH es accesible, arreciando los ataques de fuerza bruta y es muy común que uno de los usuarios de los que prueben contraseñas sea root. El valor predeterminado es que el acceso sólo se permita mediante el uso de certificado de usuario:

- **AllowTcpForwarding**

AllowTcpForwarding yes

AllowTcpForwarding habilita la posibilidad de hacer los utilísimos túneles SSH.

- **ClientAliveInterval 180**

ClientAliveCountMax 0

Estas líneas provocan que el servidor realice una desconexión automática tras tres minutos de inactividad (180 segundos), lo cual es muy útil para evitar que se queden inadvertidamente abiertas sesiones.

- **Banner**

Banner /etc/ssh/ssh_banner

Define el fichero cuyo contenido se mostrará justamente tras la conexión y antes de la autenticación. El valor predeterminado es none, o sea, ninguno.

- **DenyUsers, AllowUsers, DenyGroups y AllowGroups**

DenyGroups vetados

AllowGroups ftpusers administradores

Las directivas DenyUsers, AllowUsers, DenyGroups y AllowGroups permiten indicar usuarios y groups a los que se les permite (exclusivamente) o niega el acceso. El usuario entrante debe cumplir con todas las directivas incluidas en el fichero. En el ejemplo, sólo podrán acceder al servidor los usuarios que

pertenezcan al grupo ftpusers o administradores y que, además, no pertenezcan al grupo vetados.

- **MaxStartups**

MaxStartups 3

MaxStartups indica el máximo número de conexiones no autenticadas simultáneas que admite el servidor⁴. En el ejemplo, por tanto, sólo podrá haber tres peticiones de autenticación produciéndose a la vez. Es útil ajustar este valor para minimizar los riesgos de ataques de fuerza bruta.

- **Match Group ftpusers**

X11Forwarding no

AllowTcpForwarding no

ForceCommand internal-sftp

La directiva Match da la posibilidad de sobrescribir la configuración global para las conexiones que cumplan los criterios especificados⁵. Por ejemplo, en este caso, a los usuarios que pertenecen al grupo ftpusers sólo se les permite el uso del servidor para transferir ficheros⁶.

Ver también Si el servidor SSH se expone en internet, no tardará en recibir ataques de fuerza bruta que persiguen el acceso a la máquina. Siempre es más que recomendable configurar algún mecanismo que los inutilice.

4 Clientes SSH

Por clientes del servidor entenderemos bajo este epígrafe sólo aquellos destinados a la administración interactiva del servidor.

4.1 OpenSSH

Es el cliente habitual en sistemas unix. El modo básico de usarlo es el siguiente:

```
$ ssh usuario_remoto@servidor
```

es decir, indicar el usuario de que se dispone en el servidor y el nombre o la ip del mismo. Si se omite el nombre de usuario, se tomará el mismo que el del usuario local. La consecuencia de esta orden es que se abrirá una consola interactiva (una sesión de la shell predefinida para tal usuario) en el servidor, tal como si hubiéramos ingresado en la máquina local. Es posible añadir argumentos a la orden, por ejemplo:

4.1.1 Iniciar sesión en un servidor con un puerto diferente

De forma predeterminada, el demonio SSH en un servidor se ejecuta en el puerto 22. Su cliente SSH asumirá que este es el caso al intentar conectarse. Si su servidor SSH está escuchando en un puerto no estándar (esto se demuestra en una sección posterior), deberá especificar el nuevo número de puerto cuando se conecte con su cliente.

Puede hacer esto especificando el número de puerto con la -p opción:

```
ssh -p port_num username@remote\_host
```

```
$ ssh -p443 usuario_remoto@servidor
```

para conectarse al puerto 443 en vez de al 22, como es habitual. Es posible también ejecutar un comando en el servidor sin llegar a usar el nodo interactivo simplemente añadiéndolo a continuación:

4.1.2 Ejecución de un solo comando en un servidor remoto

Para ejecutar un solo comando en un servidor remoto en lugar de generar una sesión de shell, puede agregar el comando después de la información de conexión, así:

```
ssh username@remote_host command_to_run
```

```
$ ssh usuario_remoto@servidor ls -l
```

Si se prefiere también incluir la orden entre comillas:

```
$ ssh usuario_remoto@servidor "ls -l"
```

Advertencia Las comillas cobran importancia cuando queremos evitar que el bash local haga interpretaciones de la orden (p.e. expanda), porque queremos que estas las haga el servidor remoto.

Este cliente permite incluir configuración dentro de `/etc/ssh/ssh_config` (configuración general) y `~/.ssh/config` (configuración personal del usuario). Como en el caso de la configuración del servidor, pueden usarse directivas globales (que afectan a todas las conexiones) y particulares con `Match` o, en este caso, también `Host`. Por ejemplo:

```
# Configuración en ~/.ssh/config
ServerAliveInterval 120
ServerAliveCountMax 0

Host zipi
    Hostname  iescdl.es
    User      perico
```

Port	443
------	-----

En esta configuración hemos incluido dos directivas globales que fuerzan la desconexión del cliente tras dos minutos de inactividad. Son simétricas a las ya vistas en el servidor, y en este caso el que se desconecta es el cliente y no el servidor. Son útiles si queremos provocar desconexiones automáticas y en el servidor no se ha hecho configuración alguna al respecto.

Además se ha definido la conexión a una máquina, de manera que la orden:

```
$ ssh zipi  
es equivalente a:  
$ ssh -p443 perico@iescdl.es
```

Lo que no puede hacerse es añadir también la contraseña, porque openssh obliga a que esta se facilite siempre de manera interactiva. Para evitarlo, es necesario recurrir a otras estrategias.

4.2 Putty

Es el cliente más habitual en los sistemas windows. Es efectivo, no requiere siquiera instalación y, además, soporta otros protocolos como telnet o conexión serial, lo que lo hace utilísimo en muchos casos.

5 Autenticación de clave pública

Las técnicas de cifrado asimétrico se usan en el protocolo SSH para la autenticación tanto de servidor como de cliente. En el caso del segundo, también es posible la autenticación con contraseña, que es justamente la que hemos visto hasta ahora.

Cuando se usa autenticación con clave pública hay dos posibilidades:

- Una solución basada exclusivamente en el uso de claves asimétricas
- Otra que, sobre la base de la anterior, genera certificados digitales a partir de las claves públicas utilizando una infraestructura PKI.

6 Autenticación de en ssh mediante clave pública exclusivamente.

6.1 Identificación del servidor

Ya se ha mencionado anteriormente que para evitar ataques man-in-the-middle mediante la suplantación de la identidad del servidor, éste envía al cliente su clave pública. En la primera conexión aceptar la clave pública de un servidor es un acto de fe, a menos que se haya obtenido esa clave pública de algún modo seguro. Sea como sea, el cliente, por su parte, puede almacenar las claves públicas de todos los servidores a los que se conecta, de modo que si se produce una segunda conexión compara la clave pública que le vuelve a enviar el servidor con la que previamente tiene almacenada. En realidad lo que compara es el fingerprint o huella dactilar de la clave pública, una sucesión de pares de números hexadecimales que se obtiene a partir de ella: si coinciden, el cliente se fiará de que el servidor es quien dice ser; si no coinciden, entonces sospechará que hay una suplantación y, o bien avisará (caso de putty), o bien, se negará en rotundo a conectar hasta que no se elimine del cliente la clave pública antigua (caso del cliente de openssh).

Del párrafo anterior, hay dos ideas que es necesario remarcar y que se desglosarán bajo los siguientes epígrafes:

1. El servidor tiene una pareja de claves pública/privada que le sirven para identificarse ante el cliente (también sirven para cifrar la clave simétrica, pero esto importa poco a los efectos de esta discusión).
2. El cliente almacena las claves públicas de los servidores para comprobar las identidades de éstos cuando intenta una conexión.

6.2 Claves del servidor

Se ha afirmado que el servidor tiene una pareja de claves. Esto no es del todo cierto, en realidad, las versiones modernas del servidor openssh tiene varias parejas de claves, cada una de cuales usa un algoritmo diferente de cifrado¹. Estas claves están almacenadas en el directorio /etc/ssh:

```
$ ls -l /etc/ssh/ssh_host_*  
/etc/ssh/ssh_host_ecdsa_key  
/etc/ssh/ssh_host_ecdsa_key.pub  
/etc/ssh/ssh_host_ed25519_key  
/etc/ssh/ssh_host_ed25519_key.pub  
/etc/ssh/ssh_host_rsa_key  
/etc/ssh/ssh_host_rsa_key.pub
```

Los ficheros de extensión .pub son los que almacenan las claves públicas. Paralelamente, en el fichero de configuración debe indicarse que estas serán las claves de identificación2:

```
$ grep -w HostKey /etc/ssh/sshd_config
#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key
```

Cuál de estas parejas se use para la validación depende del cliente: él le indicará al servidor por orden que algoritmos prefiere y el servidor usará el primero que tenga disponible. Se ha dicho también que en realidad lo que se compara es el fingerprint obtenido a partir de la clave pública. De hecho, la primera vez que se conecta el cliente al servidor, se encontrará con esto:

```
pepe@cliente$ ssh usuario@servidor
The authenticity of host 'servidor (192.168.1.11)' can't be established.
ECDSA key fingerprint is SHA256:v0ef4Mo/0obxdDeS9iNIZ5+kQpMB+krMf9XTedRMUbE.
Are you sure you want to continue connecting (yes/no)
```

Aceptarlo implicará almacenar en el cliente tal fingerprint. Tal huella digital es posible obtenerla manualmente gracias a ssh-keygen:

```
$ ssh-keygen -l -f /etc/ssh/ssh_host_ecdsa_key.pub

256 SHA256:v0ef4Mo/0obxdDeS9iNIZ5+kQpMB+krMf9XTedRMUbE root@servidor (ECDSA)
```

que obviamente coincide con el fingerprint que observó pepe desde el cliente. El primer número (256) indica la longitud de la clave.

Advertencia Las claves identifican al servidor, así que es mejor no cambiarlas a menos que las sospechemos comprometidas. Si es absolutamente necesario, pueden redefinirse así:

```
# ssh-keygen -t ecdsa -f /etc/ssh/ssh_host_ecdsa_key -N ""
```

que crea una clave ECDSA en el fichero apropiado sin contraseña (esto último necesario, porque el servidor debe ser capaz de usarlas sin pedir la contraseña a nadie). La orden debería repetirse para todos los algoritmos, así que podríamos hacer:

```
# for algo in rsa ecdsa ed25519; do ssh-keygen -t $algo -f /etc/ssh/ssh_host_${algo}_key -N ""; done
```

6.3 Claves en el cliente

Como se ha adelantado, los clientes almacenan los fingerprints de las claves públicas de los servidores a los que se han conectado previamente dentro del fichero `~/.ssh/known_hosts`. Ahora bien:

```
pepe@cliente$ ssh-keygen -l -f ~/.ssh/known_hosts
256          SHA256:v0ef4Mo/0obxdDeS9iNIZ5+kQpMB+krMf9XTedRMUbE      |1|
4WnmUKMOBGnrOtOHytjLzB8cKFE=|q7chrsGIIRUBS2S1GlpzA0vDbjo= (ECDSA)
256SHA256:v0ef4Mo/0obxdDeS9iNIZ5+kQpMB+krMf9XTedRMUbE
|1|l4/bYn6a2ImKN27oCuWd3wb9M8o=|b5i6kcIRct1K1U9EBVH4PgfoJJU= (ECDSA)
```

si comprobamos los fingerprints almacenados no seremos capaces de saber a qué servidor corresponde tal fingerprint. Esto se hace así por privacidad, esto es, por evitar que otros usuarios sean capaces de conocer a qué servidores nos conectamos³. El comportamiento puede modificarse cambiando el valor de la directiva `HashKnownHosts` en la configuración del cliente. Por tanto, si añadimos como directiva global a `~/.ssh/config`:

HashKnownHosts no

se almacenarán las direcciones en claro:

```
pepe@cliente$ rm -f ~/.ssh/known_hosts
pepe@cliente$ ssh usuario@servidor
[...]
pepe@cliente$ ssh-keygen -l -f ~/.ssh/known_hosts
256  SHA256:v0ef4Mo/0obxdDeS9iNIZ5+kQpMB+krMf9XTedRMUbE  servidor,192.168.1.11
(ECDSA)
```

En cualquier caso, aunque estén ofuscados los nombres, puede buscarse si hay ya alguna entrada para un servidor:

```
$ ssh-keygen -F servidor
```

A partir de ahora, cada vez que se conecte a servidor, se confirmará que el fingerprint de la clave proporcionada por este coincide con la almacenada. Si no es así, ocurrirá esto:

```
pepe@cliente$ ssh usuario@servidor
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!    @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ECDSA key sent by the remote host is
SHA256:IWvZBm4stPCufdULUdUtKGkk7P/nNhDsf1R5XDUANtM.
```

Please contact your system administrator.

```
Add correct host key in /home/josem/.ssh/known_hosts to get rid of this
message.
Offending ECDSA key in /home/josem/.ssh/known_hosts:1
  remove with:
  ssh-keygen -f "/home/josem/.ssh/known_hosts" -R "servidor"
ECDSA host key for servidor has changed and you have requested strict
checking.
Host key verification failed.
```

Que no coincidan las claves puede suponer una suplantación de identidad... o que las claves cambiaron en el servidor por alguna razón. Si la razón es esta segunda, la solución está incluida en el mensaje de advertencia, y es borrar el fingerprint asociado al servidor:

```
pepe@cliente$ ssh-keygen -R servidor
# Host servidor found: line 1
/home/josem/.ssh/known_hosts updated.
Original contents retained as /home/josem/.ssh/known_hosts.old
```

Nota Putty también almacena el fingerprint del servidor, pero a diferencia de éste, no rechaza la conexión cuando detecta un cambio en la clave, sino que advierte del peligro de seguridad y permite aceptar (o no) la nueva clave.

6.4 Identificación del cliente

Obviamente, nos referimos a su identificación mediante clave y no mediante contraseña. Por supuesto, la identificación exige primero la creación de las claves y que el servidor acepte este tipo de identificación:

```
$ grep -w PubkeyAuthentication /etc/ssh/sshd_config  
#PubkeyAuthentication yes
```

6.4.1 Cliente openssh

En este caso, la generación es sencilla. La orden:

```
ssh-keygen
```

Que genera un par de claves con el sistema de encriptación por defecto del sistema.

Este procedimiento ha generado un par de claves SSH RSA, ubicado en el .sshdirectorío oculto dentro del directorio de inicio de su usuario. Estos archivos son:

- ~/.ssh/id_rsa: La clave privada. ¡NO COMPARTE ESTE ARCHIVO!
- ~/.ssh/id_rsa.pub: La clave pública asociada. Esto se puede compartir libremente sin consecuencias.

O especificar un tipo de encriptación diferente.

```
$ ssh-keygen -t ecdsa -C "Comentario alusivo a la clave"
```

generará una clave de tipo ECDSA y, como fichero, se escogerá el predeterminado para este tipo de clave (~/.ssh/id_ecdsa y ~/.ssh/id_ecdsa.pub, para la clave pública). Si no se altera la configuración, en el momento en que intentemos conectar a cualquier servidor se intentará usar esta clave para la autenticación. Tras generar la pareja de claves se nos pide una clave de paso, que no es más que una contraseña con la que cifrar la clave privada. Fijarla obliga a escribir tal contraseña antes de usarla y nos asegura ante un hurto de las claves.

6.4.1.1 Eliminar o cambiar la frase de contraseña en una clave privada

Si ha generado una frase de contraseña para su clave privada y desea cambiarla o eliminarla, puede hacerlo fácilmente.

Nota : Para cambiar o eliminar la frase de contraseña, debe conocer la frase de contraseña original. Si ha perdido la contraseña de la clave, no hay ningún recurso y tendrá que generar un nuevo par de claves.

Para cambiar o eliminar la contraseña, simplemente escriba:

```
ssh-keygen -p
```

6.4.2 Copia de su clave SSH pública a un servidor con SSH-Copy-ID

Es obvio, que no basta con generar la clave en el cliente: en el servidor debe alojarse la clave pública para que, cuando se produzca la comunicación, pueda asegurarse que quien se conecta desde el cliente es el propietario de la clave privada correspondiente. Para ello, puede hacerse:

```
$ ssh-copy-id usuario@servidor
```

que subirá la clave al fichero del servidor que almacena las claves públicas (`~/.ssh/authorized_keys`)⁵. A partir de ahora, al intentar ingresar en el servidor, se usará en primer término la identificación mediante el certificado y se nos pedirá la clave de paso (ya que hay que usar la clave privada) o nada, si no se introdujo contraseña al generar las claves.

6.4.3 Copia de su clave SSH pública a un servidor sin SSH-Copy-ID

Si no tiene la `ssh-copy-id` utilidad disponible, pero aún tiene acceso SSH basado en contraseña al servidor remoto, puede copiar el contenido de su clave pública de una manera diferente.

Puede generar el contenido de la clave y canalizarlo al `ssh` comando. En el lado remoto, puede asegurarse de que el `~/.ssh` directorio exista y luego agregar el contenido canalizado al `~/.ssh/authorized_keys` archivo:

```
cat ~/.ssh/id_rsa.pub | ssh username@remote_host "mkdir -p ~/.ssh && cat >>
~/.ssh/authorized_keys"
```

Se le pedirá que proporcione la contraseña de la cuenta remota:

7 Utilidades adicionales

7.1 Transferencia de ficheros

El protocolo SSH permite también la transferencia de ficheros entre cliente y servidor. Aunque puede configurarse de modo que sustituya perfectamente a un servidor FTP, por ahora nos limitaremos a usar la configuración predeterminada y ver los clientes que nos permiten esta tarea.

La primera forma de aprovechar las posibilidades de transferencia es usar una herramienta de copia remota (scp proporcionada por openssh o pscp.exe de la suite de putty). El modo de usarlo es muy semejante al de la orden cp del mundo unix:

```
$ scp /path/local/fichero.txt usuario@servidor:path/en/el/servidor
```

es decir, un origen (en este caso un fichero del cliente) y un destino que se expresa como al conectarnos con el cliente ssh seguido de dos puntos. Si la ruta en el servidor es relativa se sobreentiende que lo es respecto al directorio personal del usuario. Si no se expresa ruta alguna, el fichero se copiará en el directorio personal. En el ejemplo, el origen es local y el destino remoto, por lo que la consecuencia es que subimos el fichero al servidor. Para descargar no hay más que escoger un origen remoto y un destino local1:

```
$ scp usuario@servidor:fichero_remoto.txt .
```

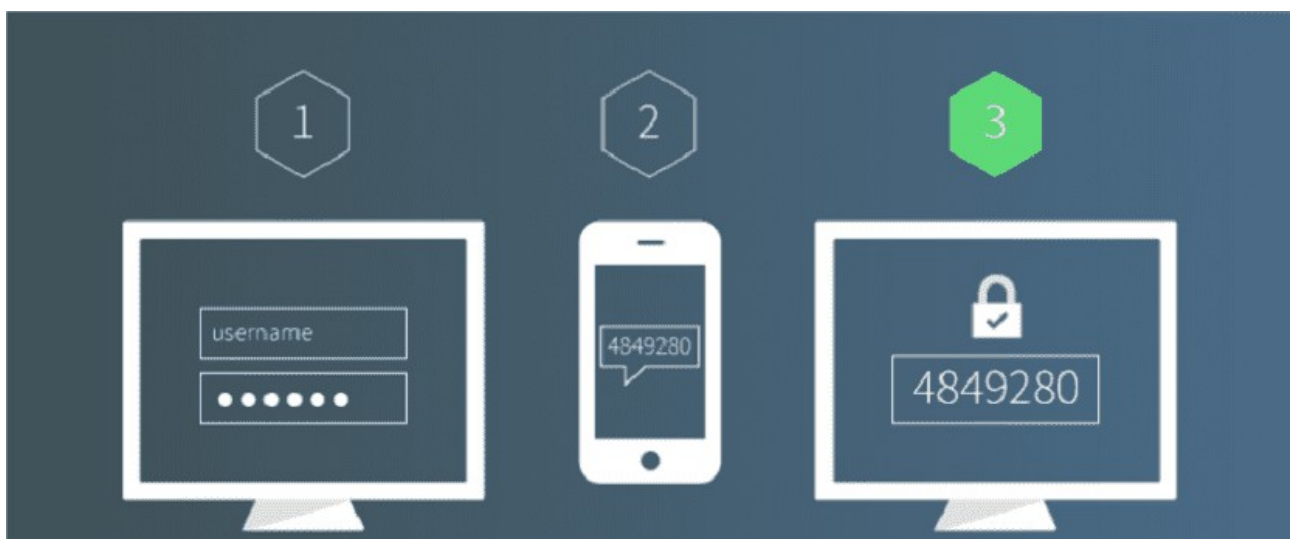
La segunda forma de transferir ficheros es usar un cliente [s]ftp, similar al cliente del FTP tradicional. openssh proporciona sftp, y putty su equivalente psftp

```
$ sftp usuario@servidor
sftp> put fichero_local
sftp> get fichero_remoto
```

Algunos clientes gráficos de FTP como filezilla, soportan también el protocolo sFTP.

Nota Tanto scp como sftp hacen uso de la configuración contenida en ~/.ssh/config, por lo que podremos aprovechar las conexiones con nombre que hayamos creado en él.

8 Autenticación de dos factores en SSH



La autenticación de dos factores (2FA) no es algo exclusivo que se pueda utilizar en las redes sociales o en alguna otra web. Pues esta medida de seguridad también puede ser implementada dentro de un sistema operativo.

Es por ello que el día de hoy veremos la forma de cómo implementar la autenticación de dos factores en SSH en Ubuntu y derivados utilizando el conocido Autenticador de Google el cual aumentará considerablemente la seguridad de su servidor OpenSSH.

Normalmente, solo necesita ingresar una contraseña o usar la clave SSH para iniciar sesión en su sistema de forma remota.

La autenticación de dos factores (2FA) requiere que se ingresen dos datos para iniciar sesión.

Por lo tanto, también deberán de ingresar una contraseña de un solo uso basada en el tiempo para iniciar sesión en su servidor SSH.

Esta contraseña de un solo uso se calcula utilizando el algoritmo TOTP, que es un estándar IETF.

8.1 Instalación y configuración de Google Authenticator

El primer paso que vamos a realizar es la instalación de Google Authenticator en nuestro sistema, por lo que vamos a abrir una terminal en el sistema (esto lo pueden hacer con la combinación de teclas "Ctrl + Alt + T) y en ella vamos a teclear el siguiente comando:

```
sudo apt install libpam-google-authenticator
```

8.2 Configurar SSH

Para hacer que SSH use el módulo PAM del Autenticador de Google, agregue la siguiente línea al /etc/pam.d/sshd archivo justo después de **@include common-auth** :

```
@include common-auth  
#One-time password authentication via Google Authenticator  
auth required pam_google_authenticator.so
```

Modificar **/etc/ssh/sshd_config**:

cambie ChallengeResponseAuthentication de no a yes, por lo que esta parte del archivo se ve así:

```
ChallengeResponseAuthentication yes
```

Ahora necesitas reiniciar el sshd demonio usando:

```
sudo systemctl restart sshd.service
```

8.3 Configurar la autenticación

Ya realizada la instalación vamos a ejecutar la aplicación recién instalada con el siguiente comando:

```
google-authenticator
```

Al ejecutar este comando lo que vamos a realizar es asignar una clave secreta y esta nos preguntara si queremos utilizar los tokens basados en el tiempo lo que responderemos que sí.

En una terminal, ejecute el comando google-authenticator.

Te hará una serie de preguntas, aquí tienes una configuración recomendada:

```
Hacer tokens "base de tiempo" ": sí
Actualizar el .google_authenticatorarchivo: sí
No permitir usos múltiples: sí
Aumentar el límite de tiempo de la generación original: no
Habilitar la limitación de velocidad: sí
```



Es posible que haya notado el código QR gigante que apareció durante el proceso, debajo están sus códigos de emergencia para usar si no tiene acceso a su teléfono: escríbalos en un papel y guárdelos en un lugar seguro.

Eso es todo. Ahora, abramos Google Authenticator y agreguemos nuestra clave secreta para que funcione.

8.4 Instalación de la aplicación Google Authenticator en el móvil

A continuación, deberá instalar la aplicación Google Authenticator en su teléfono inteligente. Una vez instalado, abra Google Authenticator en su móvil y escanee el código QR anterior. Una vez escaneado el código QR, debería ver una contraseña de un solo uso de seis dígitos en su teléfono, como se muestra a continuación:



Esta contraseña caducará en 30 segundos, por lo que deberá abrir la aplicación Google Authenticator y ver una nueva contraseña para iniciar sesión en el servidor Ubuntu a través de SSH.

También puede ver la clave secreta, el código de verificación y el código de emergencia en el resultado anterior. Se recomienda guardarlo en un lugar seguro para su uso posterior.

Su nueva clave secreta es: 4XXQ2QA6FRV72SDM2JWSZTQJE4

Su código de verificación es 423832

Sus códigos de emergencia para raspar son:

96469497

84504151

61871048

79931657

71052210

8.5 Pruebe la autenticación de dos factores

En este punto, su servidor SSH ahora está configurado con autenticación multifactor. Es hora de conectarlo y probarlo.

En el sistema remoto, abra su terminal e inicie sesión en su servidor a través de SSH como se muestra a continuación:

```
ssh root @ ssh-servidor-ip
```

Se le pedirá que proporcione la contraseña del sistema y el código de verificación generado por Google Authenticator. Proporcione la contraseña de su sistema que se muestra en la aplicación Google Authenticator y presione Enter para iniciar sesión en el servidor como se muestra a continuación:

```
vyom@newpc:~/Project/Rosehosting/New/New/New$ ssh root@
Password:
Verification code:
Welcome to Ubuntu 20.04 LTS (GNU/Linux 5.4.0-29-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

Last login: Thu Feb 11 12:29:27 2021 from 223.184.131.131
root@ubuntu2004:~#
```